From Instruments to Investigations

A policy and design paper for a Software-Defined STEM Laboratory

UNO Q Spirit as a reference instrument · Design edition 1.0 · 25 October 2026

Status: Proposed architecture and operating policy. The UNO Q Spirit implementation is an engineering preview; this paper describes a direction, not a claim that the complete laboratory already exists.

Uno Q Spirit
EMBEDDED LABORATORY INSTRUMENTATION

> **A software-defined laboratory is a shared, programmable path from a question about the physical world to a safe experiment, a traceable measurement, and an understandable conclusion. Its defining feature is not a particular board or dashboard. It is the ability to combine replaceable instruments, explicit experiment procedures, human judgment, and inspectable software without making the science disappear behind automation.**

Executive summary

The Software-Defined Laboratory (SDL) can grow from the existing UNO Q Spirit instrument into an ecosystem that serves multiple curricula and hardware tiers. The current project already demonstrates a useful split: an STM32 endpoint owns sensor interaction; Linux hosts experiment logic, records, and a browser dashboard; and a Bridge carries structured status and commands. It also demonstrates an essential discipline: synthetic values, raw sensor counts, calibrated values, and verified physical results must never be presented as interchangeable.

This paper proposes that SDL be organized around **experiments and capabilities**, rather than around a single board or monolithic app. An experiment states its question, required measurements, procedure, acceptance checks, and record format. Instruments advertise the capabilities they actually provide. A host-side orchestrator checks that the requested procedure fits the connected equipment, obtains the required human approvals, records a

provenance-rich run, and keeps a visible audit trail. The endpoint remains responsible for bounded, local acquisition and safe device behavior; the host coordinates experiments, user interaction, storage, and optional analysis.

The educational anchor is the user's Core 12 lineage and its current canonical successor, **CORE 10**. Its exemplars show why SDL must support more than generic data logging: Kitchen Inquiry Lab asks learners to account for microwave input energy and water heating; One Cup Chemistry uses conductivity, color, and other measurements to reason about mixtures and reactions; NOTHING IS FREE / Personal Energy Laboratory follows energy through batteries, motors, robots, heat, and daily devices. The New Nugget physical-computing lessons add a complementary route: learners can move from a named action to macros and then to the actual hardware register operation. These examples require reusable instruments, but they also require learning paths that keep the phenomenon and its assumptions visible.

The recommended program is incremental. First qualify safe, repeatable instrument modules and common data contracts. Then publish experiment definitions, dashboards, and records that can be replayed or inspected. Add orchestration only after physical validation, permissions, abort behavior, calibration, and evidence handling are in place. Autonomous actions must be bounded, observable, reversible where possible, and subject to explicit human authority.

1. Purpose and policy position

1.1 Purpose

This document defines an architectural direction and operating policy for combining the projects in the SDL ecosystem into a practical software-defined physical-computing and STEM laboratory. It is intended for educators, instrument builders, programmers, maintainers, and future contributors. It describes how shared instruments and software can support multiple labs without making any one curriculum or hardware platform a requirement for all the others.

1.2 Policy position

SDL should:

1. **Begin with the scientific question.** Choose hardware because it answers a measurement or control need.
2. **Expose the measurement path.** Keep sensors, units, ranges, calibration, transformations, and uncertainty visible.
3. **Separate instrument behavior from experiment logic.** Reuse qualified instruments across experiments; keep experimental procedures in versioned definitions.
4. **Keep a human in control of consequential actions.** Automation may assist, schedule, and analyze, but authority to energize hazardous apparatus, change operating conditions, or accept conclusions must be explicit.

5. **Make evidence reproducible.** Preserve the experiment definition, software and firmware versions, instrument identity, calibration state, raw observations, operator actions, and derived results together.
6. **Build from inexpensive and inspectable parts.** Keep a low-cost path available and teach how it works; provide more capable tiers when they serve the investigation.
7. **Label maturity honestly.** Mark content as historical, proposed, implemented, tested in simulation, bench-tested, or validated for its stated use.

1.3 What “software-defined” means here

The experiment's sequence, sampling conditions, processing, and report can be selected or changed in software without redesigning every instrument. The physical world is still accessed through a real sensor, actuator, fixture, and safety boundary. Software definition does not mean software can make an inadequate sensor accurate, remove electrical risk, or substitute a simulation for evidence.

2. Educational foundation: Core 12 lineage and current CORE 10

The historical **Core 12** concept is part of the project's intellectual lineage. The current curriculum root is named **CORE 10** in the published repository; the earlier name should be preserved when discussing historical material, not silently treated as the current canonical curriculum. SDL is the reusable measurement and computing infrastructure. CORE 10 supplies bounded investigations that give that infrastructure purpose. New Nugget teaches the machinery of computing; the SDL instrument layer lets learners apply it to physical questions.

The curriculum's teaching principles provide useful design tests for software and instruments:

- **Experience Before Equation:** let learners observe a phenomenon, then build a model and compare it with data.
- **Need Before Technology:** introduce a sensor, script, dashboard, or AI helper when the question needs it.
- **Not talk, do:** provide a complete action a learner can perform and a result they can examine.
- **How much? Enough?:** measure a quantity, then judge whether precision and evidence are adequate for the purpose.

2.1 Three representative experiment families

Curriculum exemplar	Physical question	SDL services that help	What learners should still see
Kitchen Inquiry Lab: microwave water heating	How much electrical input energy accompanies a measured temperature rise in water? How does the estimate compare with `Q = mcΔT`?	Temperature probe, timing, optional power/energy instrument, experiment timer, CSV and graph, calculation notebook	Water mass, start/end temperature, heating interval, input energy, heat loss, assumptions, uncertainty, and the distinction between electrical input and heat gained by the water
One Cup Chemistry: conductivity / color / crystallization	How does concentration, temperature, or reaction state change a measurable electrical or optical signal?	Reusable conductivity cell, temperature, optical/color sensor such as AS7343-class hardware, blank/reference capture, sample labels, repeatable procedure	Sensor response is not itself chemical identity; rinse/contamination controls, illumination and geometry, blanks, raw channels, calibration and comparison standards
NOTHING IS FREE / Personal Energy Laboratory	How do battery condition, load, motion, heat, or operating time relate to energy use?	Voltage/current/temperature/time/distance acquisition, robot or motor endpoint, run plan, energy integration, archive and comparison	Battery chemistry and limits, sign conventions, units, power integration, mechanical/environmental losses, repeat trials and safe stop conditions

These are design exemplars, not claims that all required instruments or lesson packages are currently implemented in UNO Q Spirit. The first releases should support a narrow, well-qualified experiment end to end, then extend the shared instrument layer as other labs need it.

2.2 Computing concepts as a visible learning ladder

The New Nugget / MSP430 learning sequence offers a model for progressive disclosure. A learner can first make an LED respond, name the action (`LED_ON`), inspect the general bit macro, and finally examine the actual register write. SDL should preserve this same learning ladder at system level:

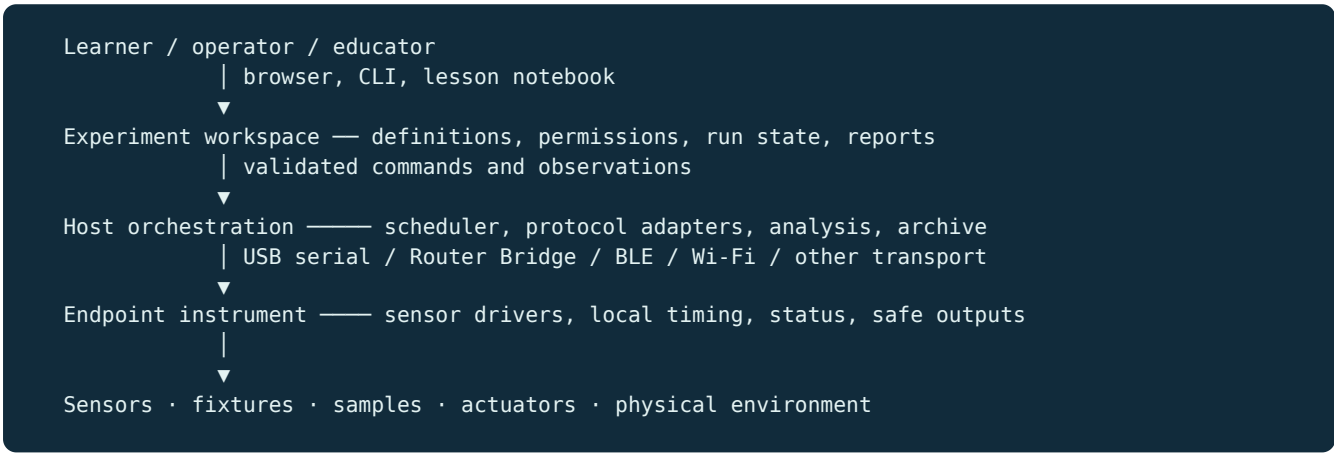
1. Observe and run a working experiment.
2. Inspect the sensor value, state, unit, and timestamp.
3. Open the procedure and see its steps and checks.
4. Inspect the host command, endpoint protocol, and transform.

5. Trace the low-level driver or GPIO operation when the learner is ready.

The dashboard should make the instrument accessible without hiding how the reading was produced. Educational “simple mode” can reduce clutter, while an adjacent “show details” path reveals the implementation.

3. Reference architecture

SDL is a layered architecture that can be implemented with different boards and transports. UNO Q Spirit is one capable reference tier, not the only permitted endpoint.



3.1 Endpoint and transport

An endpoint is responsible for talking to sensors and actuators, applying bounded local timing, reporting health and measurement state, and entering a defined safe state on reset or communication loss where the hardware permits. A simple Arduino and USB serial connection is a valid entry point. A UNO Q can keep acquisition on its STM32 MCU while Linux runs Python, network services, the experiment interface, and records. Pico W, ESP32-class boards, micro:bit, BLE UART, Qwiic/STEMMA QT accessories, and future endpoints can join through adapters when their capability and safety evidence is documented.

The transport should not be confused with the instrument contract. A sensor may be attached through a STEMMA/Qwiic connector, header pins, or a Spirit shield; its capability and interface contract should remain portable. Optional accessories should remain optional. A learner should be able to run an AS7343 color/light exploration using a compatible breakout and a supported bus without needing the Spirit shield.

3.2 Host bridge and experiment workspace

The host side offers a stable interface across endpoint-specific transports. It validates messages, normalizes timestamps and units, tracks connection and instrument health, routes only permitted commands, and records both the raw observation and any derived result. The experiment workspace presents the procedure, operator prompts, live data, controls, run status, and archive. It must make active mode and data origin explicit: live, synthetic, replayed, or simulated.

The current UNO Q Spirit dashboard is an early host workspace: it polls Bridge status, stores snapshots locally, and exports CSV. Its present Start/Stop controls manage Linux-side recording; they do not start or stop MCU acquisition. This distinction is a product behavior that future experiment pages should retain or revise explicitly.

3.3 Optional analysis and AI collaboration

Analysis may range from a spreadsheet formula to Python fitting, graph comparison, or an AI assistant. The analysis layer should work from recorded, versioned inputs and produce outputs that can be reviewed. An AI may suggest a measurement range, generate a plot, compare a result with a model, help explain an error, or propose a next experiment. It must state when it is using synthetic data, identify assumptions, and avoid converting a suggestion into a physical command without an authorized human action and safety check.

4. Experiment as a versioned software object

An experiment should be a declarative, reviewable package rather than an opaque script. A first schema can use YAML or JSON and reference an instrument capability by name, leaving device details to a binding layer.

```
id: kitchen.water-heating.energy-balance
version: 0.1.0
question: How does measured input energy compare with water heating?
measurements:
  - quantity: temperature
    unit: degC
    cadence: 1 s
  - quantity: elapsed_time
    unit: s
    source: host_clock
procedure:
  - prompt: Measure and record the water mass.
  - prompt: Place the temperature probe safely before heating.
  - action: record_start_temperature
  - action: start_timed_interval
  - prompt: Stop heating at the selected interval and record final temperature.
analysis:
  - derived: water_heat_gain
    expression: mass_kg * 4184 J/(kg*K) * delta_temperature_K
  - note: Compare with measured electrical input; discuss heat loss.
human_approval:
  required_before: [start_heating]
limits:
  max_run_time_s: 120
```

The example is illustrative; expressions and action semantics need a formal schema and validator before execution. The experiment object should include:

- learning goals and prerequisite knowledge;
- question, hypothesis, controlled variables, and procedure;
- required instrument capabilities and acceptable ranges;

- raw data fields, canonical units, and expected status values;
- calibration and reference requirements;
- operator gates, run limits, abort rules, and post-run checks;
- analysis steps and derived-value definitions;
- version, author, license, citations, and known limitations.

5. Data, provenance, and reproducibility policy

5.1 Record both observation and interpretation

Keep the endpoint's raw value and status, host receive time, endpoint time or counter where available, units, calibration identifier, instrument identity, and any transformed value. Do not overwrite raw data during cleaning. If an observation is missing or invalid, record its state rather than inserting zero. Derived values should name their formula, parameters, source channels, and software version.

For example, an optical sensor's 18 raw channels are not wavelength-calibrated irradiance. A load-cell count is not mass until a calibration and its conditions are applied. A synthetic temperature is not a physical reading. Status and data origin are part of the data, not optional UI decoration.

5.2 Run manifest

Each run should have a machine-readable manifest tying together:

- experiment ID and version, plus a frozen copy or content hash of the procedure;
- endpoint identifier, board/firmware build, host software version, and transport;
- sensor model, serial or instance ID where available, wiring/profile, and capabilities;
- calibration references, date, method, and validity status;
- operator, course or group context, start/stop times, and notes;
- sample rows, alarms, command history, pauses, faults, and abort reason;
- analysis version and derived outputs.

CSV remains valuable for immediate use, but should be accompanied by a manifest (for example, JSON) so the records retain context outside the dashboard. Longer term, the archive can expose a stable API and export formats without making a specific database a user-facing contract.

5.3 Evidence maturity labels

Every published artifact and dashboard result should use a consistent evidence vocabulary:

Label	Meaning
Proposed	Architecture or procedure not yet implemented.
Implemented	Source code or document exists; operation may not have been demonstrated.
Simulation-tested	Software path checked with synthetic, fixture, or mock data.
Bench-tested	Named hardware and stated procedure exercised on a bench; evidence recorded.
Validated for purpose	Acceptance criteria, calibration, repeatability, and limitations reviewed for a defined educational or measurement use.

One component's maturity does not transfer automatically to another. A compiled sketch does not validate wiring; a passing mock does not validate sensor accuracy; a browser screenshot does not validate the physical experiment.

6. Safety, permissions, and bounded autonomy

6.1 Safety boundary

The instrument and its power interfaces must be electrically qualified for the connected board. The existing UNO Q Spirit documentation correctly treats its inherited 5 V shield as unqualified and warns against direct connection. The policy for the full SDL is the same: validate voltage domains, current limits, grounding, isolation, energy sources, thermal exposure, moving machinery, chemical handling, and sensor suitability before a procedure is approved for use. Software interlocks are a layer of defense and cannot replace electrical protection, physical guarding, supervision, or appropriate lab practice.

Each experiment definition should specify risk class, prerequisites, required supervision, energy and time limits, emergency stop behavior, allowed actuator commands, and what happens after host loss, Bridge loss, reset, sensor fault, or out-of-range values. Establish the safe behavior per apparatus; do not imply that a generic MCU stop command is an emergency-stop system.

6.2 Permission model

Separate permissions for viewing, recording, changing measurement settings, and energizing or moving apparatus. The least powerful role that supports a lesson should be the default. In classrooms, instructors define which actions learners can take. Every consequential command should include actor, target, parameters, timestamp, approval state, and result in the run record.

6.3 Autonomy ladder

Autonomy is a set of capabilities released by evidence, not a product toggle.

Stage	System role	Example	Gate to advance
0 — Observe	Display and record readings	Current Monitor tab and CSV archive	Data origin, units, state, and recording behavior clear
1 — Guide	Explain procedure and prompt the operator	Guided temperature run	Procedure review; stop/abort states tested
2 — Assist	Recommend settings or next steps; human applies them	Suggest repeat trials or narrower measurement range	Recommendations trace to data/model and are explicitly approved
3 — Bounded execution	Run a pre-approved procedure inside fixed limits	Sample at a fixed cadence; trigger low-energy indicator under supervision	Bench validation, risk review, local limit enforcement, tested abort and fault response
4 — Adaptive operation	Select among pre-approved branches from observed conditions	Adjust sampling cadence within a set range	Evidence of robust fault handling, explainable decisions, auditability, and instructor control
5 — Open-ended control	Create and execute novel physical procedures	Not part of the near-term roadmap	Not authorized by this design paper

Any future stage must keep the run inspectable and stoppable. Loss of a host connection must not leave a hazardous actuator in an undefined state. The endpoint should enforce limits locally when feasible, since network and host software cannot provide reliable real-time safety by themselves.

7. User and developer experience

7.1 User workspace

The browser dashboard should grow into an experiment workspace with clear spaces for:

- **Discover:** choose an experiment by question, available instrument, and skill level.
- **Prepare:** see wiring, materials, calibration, hazards, and preflight checks.
- **Run:** follow prompts, observe live sensor states, and see active recording/actuator state.
- **Analyze:** graph raw and processed values; compare with a model and repeat trials.
- **Review:** annotate, export, compare, and share the complete record.

Retain accessible typography, keyboard controls, legible contrast, and responsive layouts. Feedback must say what a control actually did (for example, “recording started” versus “sensor sampling started”). Data should remain useful without a network service; local operation and durable export are core requirements.

7.2 Developer contract

Hardware integrations should implement a small documented adapter contract:

- declare capability names, measurement quantities, units, ranges, cadence limits, and hardware requirements;
- report states such as `OK`, `WARMUP`, `MISSING`, `TIMEOUT`, `FAULT`, `SATURATED`, `DISABLED`, and `SYNTH` consistently;
- accept only typed, bounded, authorized commands;
- return schema-versioned observations and explicit command results;
- include self-description and diagnostic version information;
- support simulation or recorded fixtures for software development, with synthetic status propagated end to end.

Transport adapters should be independently testable from experiment procedures. Example adapters may include Router Bridge for UNO Q, USB serial for Uno/MSP430/Pico, and network/BLE links where needed. Protocol choice should follow device constraints and educational goals, not be required to match across all hardware generations.

7.3 Open and replaceable interfaces

Prefer documented schemas, exportable data, openly inspectable source, and modular instrument drivers. Keep manufacturer-specific behavior behind adapters. A teacher should be able to use an exported CSV and a published lesson without depending on a proprietary cloud account. Public contributions should include license, hardware bill of materials, safety notes, build instructions, exact validation scope, and a maintainer contact or issue path.

8. Roadmap and acceptance gates

Phase A — establish a truthful reference instrument

- Finish UNO Q Spirit firmware and Python/dashboard integration checks on the board.
- Qualify a revised 3.3 V-safe interface before attaching the legacy shield.
- Bench-test each sensor path, fault response, Bridge loss behavior, and calibration procedure.
- Record real sensor evidence separately from simulation and publish a hardware checklist.

Exit: a named sensor configuration can complete a documented run with repeatable physical readings and a complete record, with limitations stated.

Phase B — define shared instrument and experiment contracts

- Publish versioned measurement and command schemas with units and state values.
- Define capability discovery and adapters for at least two endpoint tiers.

- Create declarative experiment packages with human approval and validation.
- Add a run manifest, calibration references, and exportable records.

Exit: one instrument can be reused by at least two experiment definitions; one experiment can run on two suitable endpoint configurations with explicit capability checks.

Phase C — deliver curriculum exemplars

- Implement an end-to-end Kitchen Inquiry energy-balance experiment.
- Implement a One Cup Chemistry measurement experiment using conductivity or optical comparison, with blank/reference and contamination controls.
- Implement a Personal Energy Lab investigation using a battery-powered motor/robot or other safe load, with explicit energy accounting.
- Add the New Nugget observation-to-register teaching path where it helps learners understand how sensors and outputs are implemented.

Exit: each experiment has teacher/student materials, procedure version, dataset example, analysis notes, safety review, and acceptance evidence.

Phase D — add guided and bounded automation

- Add operator prompts, preflight checks, automatic capture, and safe stop handling.
- Test restart, disconnected sensors, invalid data, host/transport loss, and interrupted runs.
- Add optional analysis and AI collaboration that produces reviewable suggestions, not hidden physical actions.
- Pilot with educators and revise based on accessibility, instructional value, and failure reports.

Exit: bounded actions have local limits, tested stop/fault behavior, human approval policy, and complete audit records.

Phase E — grow the instrument network

- Reuse compatible peripherals through Qwiic/STEMMA QT or other documented connectors where electrically appropriate.
- Add instruments for timing/frequency, electrical energy, distance/motion, conductivity, and optical measurements based on curriculum need.
- Publish additional endpoint adapters and preserve the low-cost standalone path.
- Establish long-term maintenance, archived releases, data retention, and classroom support practices.

Exit: multiple curriculum branches share stable instrument services and records while remaining independently usable.

9. Governance and project organization

To prevent the ecosystem from becoming a collection of unrelated demos, maintain explicit ownership boundaries:

- **SDL:** endpoint/transport/host architecture, common schemas, dashboards, record and safety policy.
- **CORE 10:** the canonical curriculum hierarchy and investigations, including Kitchen Inquiry Lab, One Cup Chemistry, and NOTHING IS FREE / Personal Energy Laboratory.
- **Measured World Lab:** reusable instruments, calibration, and measurement guidance.
- **New Nugget / physical-computing curriculum:** progressive understanding of firmware, interfaces, timing, and registers.
- **UNO Q Spirit:** a reference instrument implementation and a place to validate one endpoint tier.
- **CRCL or future experiment language:** a candidate notation for laboratory intent, to be evaluated against declarative YAML/JSON and existing tools before adoption.

Each repository should link to the others rather than duplicating their canonical content. Historical Core 12 and New Nugget documents should be discoverable as history and source material, while current curriculum identifiers remain unambiguous. Before promoting a document, experiment, or hardware kit, record its review state and validation evidence in the repository that owns it.

10. Measures of progress

SDL progress should be assessed by outcomes learners and operators can verify:

1. **Question-to-evidence time:** a learner can move from a lesson question to a safely collected, exportable dataset without a bespoke software rewrite.
2. **Instrument reuse:** one qualified instrument supports multiple experiments, with no hidden dependence on a particular dashboard.
3. **Portability:** a second supported endpoint can supply the same declared measurement capability or gives a clear incompatibility result.
4. **Reproducibility:** an archived run can be interpreted from its manifest, raw values, calibration, procedure, and software versions.
5. **Truthful measurement:** learners can distinguish simulation, raw counts, calibrated measurement, and derived estimates.
6. **Safe operation:** failure and stop tests are documented for each apparatus and action class.
7. **Learning agency:** learners can inspect and modify the procedure or implementation at an appropriate depth.

8. **Equity and access:** low-cost tools, accessible controls, and offline/export workflows remain supported.

These measures should not be condensed into an unsupported single “lab quality” score. Each dimension has distinct evidence and tradeoffs.

11. Closing direction

The software-defined STEM laboratory should make it easier to ask a physical question, assemble an appropriate instrument, understand the path from sensor to conclusion, and preserve the evidence for another person to examine. The project already has one valuable vertical slice in UNO Q Spirit: an MCU/MPU division, telemetry, CLI, browser dashboard, archive, and a clear separation between simulated and physical claims. The next step is rigorous physical qualification and a shared experiment/data contract—not an assumption that a dashboard alone creates a laboratory.

The long view is a set of small, reusable, explainable instruments and experiments. A Kitchen Inquiry lesson can use the same temperature and timing services as a One Cup Chemistry exploration. A Personal Energy investigation can add voltage, current, motion, and time while reusing the run record. A learner can begin with a browser procedure, open the command, inspect the driver, and finally understand the GPIO or register operation. Humans choose the question and remain responsible for consequential decisions; software makes procedures repeatable, measurements legible, and collaboration scalable.

That is the design ambition: not to automate science away, but to make capable physical inquiry more accessible, inspectable, and reusable.

Related project references

- Software-Defined Laboratory — core architecture and reference implementation.
- CORE 10 — current curriculum root and the successor to the historical Core 12 naming.
- Measured World Lab — reusable educational instruments and measurement principles.
- New Nugget physical-computing labs — staged MCU learning, inquiry appendices, and historical Core 12 context.
- UNO Q Spirit user's and programmer's guide — current instrument operation, code structure, and known limits.
- UNO Q Spirit product specification, dashboard guide, safety notes, and validation record.